САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №4
«Развёртывание приложения на удалённом сервере»

Выполнил:

Якшин Артемий

Группа БР1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

1. Провести предварительную настройку удалённого сервера (установка необходимых зависимостей).
2. Развернуть приложение на удалённом сервере.
3. Настроить nginx в режиме обратного прокси для доступа к приложению извне.

Ход работы

1. Сервер и ограничения

Используется арендованный VPS: Ubuntu 24.04 LTS, 4 vCPU, 8 ГБ RAM, 60 ГБ SSD. В отчёте адрес сервера заменён на `<SERVER_IP>`.

Сервер **общий**: на нём уже работает другой проект. Он занимает порты 80/443, у него свои контейнеры, а firewall (ufw) пропускает снаружи только SSH. Отсюда требования к развёртыванию:

Требование	Решение
не мешать соседнему проекту	все файлы стенда — в `/opt/rb-sem2`, все docker-ресурсы с префиксом `rb-` (compose-проекты `rb-app`, `rb-edge`, сеть `rb-edge`)
не конкурировать за CPU	все контейнеры стенда привязаны к ядрам 2–3 (`cpuset: "2,3"`) и ограничены по `cpus`; ядра 0–1 всегда остаются соседям
не съесть память	у каждого контейнера лимит памяти (192 МБ на Node-сервис, 320 МБ RabbitMQ, 64 МБ nginx)
80/443 заняты	единственная публичная точка входа — nginx на порту **8088**
не открывать лишнего	ни один сервис, кроме nginx, не публикует порты; RabbitMQ и сервисы доступны только внутри docker-сетей

2. Подготовка образов к продакшену

Перед развёртыванием Dockerfile всех шести сервисов переписаны на **multi-stage** сборку (`../lab2/*/Dockerfile`):

- этап `build` (`node:22-bookworm`) — `npm ci` по lock-файлу, компиляция TypeScript, `npm prune --omit=dev`;
- этап запуска (`node:22-bookworm-slim`) — только `dist/` и production-зависимости, процесс от непривилегированного пользователя `node`, встроенный `HEALTHCHECK` по `/health`.

Node.js 20 к ноябрю 2026 г. снят с поддержки, поэтому образы переведены на Node.js 22 LTS. Для каждого сервиса сгенерирован `package-lock.json`: без него `npm install` мог собрать разные версии зависимостей на разных машинах.

Результат: образ сервиса уменьшился с ~1.8 ГБ (ЛР3, полный `node:20`) до **~440 МБ**, образ gateway весит 359 МБ.

Попутно в приложение добавлено то, что понадобится в следующих работах: эндпоинт `/metrics` (prom-client) для ЛР6/ЛР8, поддержка PostgreSQL через `DATABASE_URL` для ЛР7, Swagger UI `/api/docs/` для ДЗ9 и корректное завершение по `SIGTERM`.

3. Предварительная настройка сервера

Настройка оформлена идемпотентным скриптом `scripts/server-setup.sh`, его можно запускать повторно. Скрипт:

1. проверяет системные пакеты (`git`, `curl`, `jq`, `openssl`, `rsync`) и ставит только недостающие: `apt-get install --no-upgrade`, без обновления остальной системы;
2. проверяет Docker Engine, Compose и Buildx. На чистой машине ставит их из официального репозитория Docker, работающий Docker не переустанавливает и не перезапускает;
3. создаёт каталоги стенда `/opt/rb-sem2/{app,bin,edge}`;
4. создаёт общую docker-сеть `rb-edge`, через которую nginx ходит к gateway;
5. создаёт отдельный сборщик образов `rb-builder` (buildx, драйвер `docker-container`) с жёсткими лимитами: 1 CPU на ядрах 2–3 и 1.5 ГБ RAM, чтобы сборка не отнимала ресурсы у соседей;
6. клонирует исходники (публичный репозиторий) в `/opt/rb-sem2/src`;
7. генерирует секреты (`JWT_SECRET`, `INTERNAL_API_KEY`, пароль RabbitMQ) через `openssl rand` в `/opt/rb-sem2/app/.env` с правами `0600`. Секреты создаются на сервере и в git не попадают.

```
$ scp scripts/server-setup.sh <server>:/tmp/ && ssh <server> 'bash /tmp/server-
setup.sh'
[setup] system packages: all present (git, curl, jq, openssl, rsync)
[setup] Docker version 29.8.1, build 4a63305
[setup] Docker Compose version v5.5.1
[setup] github.com/docker/buildx v0.37.1
[setup] directories under /opt/rb-sem2 ready
[setup] docker network rb-edge ready
[setup] buildx builder rb-builder ready (cpuset 2-3, 1 CPU, 1.5 GiB)
[setup] sources: 4c62665 feat(lab4): add server setup and deploy configs
```

```
[setup] secrets generated -> /opt/rb-sem2/app/.env (0600)
[setup] done

$ docker buildx inspect rb-builder
Driver:          docker-container
Driver Options:  cpu-quota="100000" cpuset-cpus="2,3" default-load="true"
memory="1536m" cpu-period="100000"
Status:          running
```

На сервере Docker и утилиты уже были установлены, поэтому скрипт их только проверил. Первая попытка упала на опции `--driver-opt cpuset-cpus=2,3`: buildx разбирает значение как CSV, и запятая ломает разбор. Помогло экранирование `'"cpuset-cpus=2,3"'`.

4. Production-конфигурация

Файл `deploy/docker-compose.prod.yml` основан на учебном `docker-compose.yml` из ЛР3, отличия:

- **нет `ports:`** ни у одного сервиса: наружу смотрит только edge-nginx;
- секреты подставляются из `--env-file /opt/rb-sem2/app/.env` (`${JWT_SECRET:?}` — без значения compose не запустится), RabbitMQ работает с собственным логином и паролем вместо `guest/guest`;
- общие настройки вынесены в YAML-якоря: `restart: unless-stopped`, `cpuset: "2,3"`, лимиты `deploy.resources.limits`, ротация логов `json-file` (`max-size: 10m`, `max-file: 3`);
- порядок старта задаётся `depends_on: condition: service_healthy`;
- данные SQLite и RabbitMQ лежат в именованных volume'ах;
- gateway подключён к двум сетям: внутренней `rb-app` и общей `rb-edge`, где у него алиас `rb-gateway` для nginx;
- имя образа параметризовано: `${IMAGE_PREFIX:-local/rb}-<service>:${IMAGE_TAG:-latest}`. При ручном развёртывании образы собираются на сервере, а в ДЗ6 тот же файл тянет готовые образы из GHCR.

```
x-service: &service
  restart: unless-stopped
  cpuset: "2,3"
  logging: { driver: json-file, options: { max-size: "10m", max-file: "3" } }
  networks: [app]
x-node-limits: &node-limits
  deploy: { resources: { limits: { cpus: "0.50", memory: 192M } } }

services:
  auth-service:
    <<: [*service, *node-limits]
```

```
    image: ${IMAGE_PREFIX:-local/rb}-auth-service:${IMAGE_TAG:-latest}
    build: ../../lab2/auth-service
    environment:
      DB_PATH: /data/auth.sqlite
      JWT_SECRET: ${JWT_SECRET:?}
      INTERNAL_API_KEY: ${INTERNAL_API_KEY:?}
    volumes: [auth-data:/data]
  # ... catalog / reservation / review / notification / rabbitmq
  gateway:
    <<: [*service, *node-limits]
    networks:
      app: {}
      edge: { aliases: [rb-gateway] }
networks:
  app:  { name: rb-app }
  edge: { name: rb-edge, external: true }
```

5. Сборка и запуск на сервере

```
$ cd /opt/rb-sem2/src/БР1.1/Якшин\ Артемий/labs/lab4/deploy
$ docker compose -f docker-compose.prod.yml --env-file /opt/rb-sem2/app/.env build -
-builder rb-builder
 Image local/rb-gateway:latest Built
 Image local/rb-notification-service:latest Built
 Image local/rb-reservation-service:latest Built
 Image local/rb-review-service:latest Built
 Image local/rb-auth-service:latest Built
 Image local/rb-catalog-service:latest Built
real    1m46.055s

$ docker compose -f docker-compose.prod.yml --env-file /opt/rb-sem2/app/.env up -d
$ cd /opt/rb-sem2/edge && docker compose up -d        # edge-nginx (раздел 6)
$ docker buildx stop rb-builder                       # сборщик больше не нужен —
освобождаем память

$ docker ps --filter name=rb- --format "table
{{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"
NAMES                          IMAGE                                  STATUS
PORTS
rb-edge-nginx                  nginx:1.29-alpine                      Up 25 seconds
(healthy) 0.0.0.0:8088->8080/tcp, [::]:8088->8080/tcp
rb-app-gateway-1               local/rb-gateway:latest                Up 27 seconds
(healthy) 3000/tcp
rb-app-reservation-service-1   local/rb-reservation-service:latest   Up 33 seconds
(healthy) 8083/tcp
rb-app-notification-service-1  local/rb-notification-service:latest  Up 33 seconds
(healthy) 8085/tcp
rb-app-review-service-1        local/rb-review-service:latest         Up 33 seconds
(healthy) 8084/tcp
rb-app-rabbitmq-1              rabbitmq:3.13-alpine                   Up 40 seconds
(healthy) 4369/tcp, 5671-5672/tcp, ...
rb-app-auth-service-1          local/rb-auth-service:latest           Up 40 seconds
(healthy) 8081/tcp
rb-app-catalog-service-1       local/rb-catalog-service:latest        Up 40 seconds
(healthy) 8082/tcp
```

Все 8 контейнеров перешли в состояние `healthy`. В колонке `PORTS` порты сервисов указаны без `0.0.0.0:` — они только объявлены в образе и с хоста недоступны. Опубликован лишь порт 8088 у nginx.

6. nginx в режиме обратного прокси

nginx запущен отдельным compose-проектом `rb-edge` (`deploy/edge/docker-compose.yml`, образ `nginx:1.29-alpine`). Внутри контейнера он слушает 8080, на хосте этот порт опубликован как 8088. Через общую сеть `rb-edge` nginx обращается к gateway по имени `rb-gateway`. Основная часть `deploy/edge/nginx.conf`:

```
http {
  log_format main_ext '$remote_addr [$time_local] "$request" $status
$body_bytes_sent '
                      'rt=$request_time urt=$upstream_response_time
"$http_user_agent"';
  resolver 127.0.0.11 valid=10s ipv6=off;      # встроенный DNS Docker
  proxy_set_header Host              $host;
  proxy_set_header X-Real-IP         $remote_addr;
  proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
  proxy_set_header X-Forwarded-Proto $scheme;
  proxy_connect_timeout 5s;  proxy_read_timeout 30s;

  server {
    listen 8080 default_server;
    server_tokens off;
    add_header X-Content-Type-Options nosniff always;
    add_header X-Frame-Options SAMEORIGIN always;

    location = /nginx-health { return 200 "ok\n"; }
    location = /metrics      { return 404; }       # метрики сервисов наружу не
отдаём
    include /etc/nginx/rb.d/*.conf;                 # маршруты следующих работ
(/grafana/, /k8s/, /docs/)

    location / {
      set $gateway http://rb-gateway:3000;          # переменная + resolver: имя
резолвится заново,
      proxy_pass $gateway;                          # пересоздание gateway не ломает
прокси
    }
  }
}
```

Решения в конфиге:

- **`resolver` + переменная в `proxy_pass`.** Если указать апстрим константой, nginx разрешит имя один раз при старте. Тогда после пересоздания контейнера

gateway (а при автодеплое из ДЗ6 так происходит на каждый релиз) у gateway будет новый IP, и прокси начнёт отдавать 502;

- в access-лог пишется время ответа апстрима (`urt=`) — оно пригодится при разборе нагрузки в ЛР6/ЛР8;
- служебный `/metrics` закрыт, отключены версия nginx в заголовках (`server_tokens off`) и MIME-sniffing;
- каталог `conf.d/` подключается через `include`. Маршруты следующих работ добавляются отдельными файлами, основной конфиг не правится.

7. Проверка доступа извне

С рабочего компьютера (вне сервера):

```
$ curl -s http://<SERVER_IP>:8088/health -w " [%{http_code}, %{time_total}s]\n"
{"status":"ok","service":"api-gateway"} [200, 0.189729s]

$ curl -s -D - -o /dev/null http://<SERVER_IP>:8088/health | grep -iE "^(server|x-
content|x-frame)"
Server: nginx
X-Content-Type-Options: nosniff
X-Frame-Options: SAMEORIGIN

$ curl -s -o /dev/null -w "%{http_code}\n" http://<SERVER_IP>:8088/metrics      #
404 — закрыт
$ curl -s -o /dev/null -w "%{http_code}\n" http://<SERVER_IP>:8088/api/docs/    #
200 — Swagger UI

$ TOKEN=$(curl -s -X POST http://<SERVER_IP>:8088/api/v1/auth/login -H 'Content-
Type: application/json' \
    -d '{"email":"ivan@example.com","password":"securepass123"}' | jq -r .token)
# 180 символов
$ curl -s "http://<SERVER_IP>:8088/api/v1/restaurants?limit=2" | jq '.total,
[.data[].name]'
4
["La Piazza", "Sakura"]

$ curl -s -m 5 http://<SERVER_IP>:3000/health   # прямой доступ к gateway в обход
nginx
(нет ответа — порт не опубликован)
```

Что на сервере стало доступно снаружи, проверено сравнением слушающих сокетов (`ss -tlnp`) до и после развёртывания. Добавился только порт nginx:

```
$ diff listen-before.txt listen-after.txt
> 0.0.0.0:8088
> [::]:8088
```

Опубликованные Docker-порты обходят правила ufw: Docker добавляет свои цепочки iptables раньше цепочек ufw. Поэтому всё, что указано в `ports:`, сразу становится доступно из интернета. Именно поэтому в `docker-compose.prod.yml` нет ни одного `ports:`, а единственная точка входа — nginx.

Потребление ресурсов стендом сразу после запуска (`docker stats --no-stream`): все сервисы занимают 20–34 МБ при лимите 192 МБ, RabbitMQ — 127 МБ при лимите 320 МБ, nginx — 3 МБ. Всего около 300 МБ.

Как воспроизвести

```
scp scripts/server-setup.sh <server>:/tmp/ && ssh <server> 'sudo bash /tmp/server-
setup.sh'
ssh <server>
cd "/opt/rb-sem2/src/БР1.1/Якшин Артемий/labs/lab4/deploy"
docker compose -f docker-compose.prod.yml --env-file /opt/rb-sem2/app/.env build --
builder rb-builder
docker compose -f docker-compose.prod.yml --env-file /opt/rb-sem2/app/.env up -d
cp edge/nginx.conf edge/docker-compose.yml /opt/rb-sem2/edge/ && cd /opt/rb-
sem2/edge && docker compose up -d
curl -s http://127.0.0.1:8088/health
```

Вывод

Микросервисное приложение «Restaurant Booking» развёрнуто на удалённом Ubuntu-сервере. Подготовка сервера оформлена идемпотентным скриптом: он проверяет и доустанавливает зависимости, создаёт каталоги, docker-сеть и ограниченный по ресурсам сборщик образов, а секреты генерирует прямо на сервере. Образы переведены на multi-stage сборку с Node.js 22 и стали в четыре раза меньше. Production-вариант compose-файла не публикует ни одного порта, все контейнеры ограничены по CPU и памяти. Это важно, потому что сервер общий и на нём работает другой проект. Внешний доступ идёт через nginx в режиме обратного прокси на порту 8088: он передаёт клиентские заголовки, перечитывает DNS-имя апстрима, закрывает служебные эндпоинты и добавляет заголовки безопасности. Доступность API и Swagger снаружи и закрытость остальных портов проверены. Полученная схема — образы с параметризованным тегом, секреты на сервере, единая точка входа — служит основой для автодеплоя (ДЗ6) и Ansible-конфигурации (ЛР5).